

Imperfect Numbers:

- **Question Analysis:**

আমাকে একটা **Positive integer N** দেওয়া আছে।

আমাকে বের করতে হবে, **minimum possible** $|N-M|$, বা **abs(N-M)**.

যেখানে - **M** হলোঃ একটা **imperfect number**.

Imperfect number হলো সেটায়, যেটা **2** অথবা **5** দিয়ে **divisible**. কিন্তু **2** এবং **5** উভয় দিয়েই **divisible** না।

- **Observation:**

- এখানে **Constraint** কম। $1 \leq N \leq 100$
- তারমানে **Brute Force Solution possible**
- আমাদের $|N-M|$ - কে **minimize** করতে হবে। তাই **N** তো দেওয়ায় আছে, আর **M** কে **N** এর যত কাছাকাছি রাখা **possible** সেটা **try** করতে হবে, **conditions** গুলো মাথায় রেখে।
- আমার **M** , **N** থেকে বড় হতে পারে, অথবা ছোট। তো **1** থেকে **2*N** এর মধ্যেই **possible M** পাওয়া যাবে, যা **2** অথবা **5** দিয়ে **divisible**. কিন্তু **2** এবং **5** উভয় দিয়েই **divisible** না।

- **Implementation:**

- **1** থেকে **2*N** পর্যন্ত **loop** চালাব।
- **Condition** - গুলো **check** করে **possible M** বের করব।
- একটা **ans variable maintain** করব।
- যখনই $|N-M|$, কম পাব, **ans variable update** করব।

- Code:

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long

int main(){
    int tc=1;
    cin >> tc;
    while(tc--){
        int n;
        cin >> n;

        int ans = n;
        for(int i=1; i<=2*n; i++){
            if((i%2==0 || i%5==0) && i%10!=0){
                int m = i;
                ans = min(ans, abs(n-m));
            }
        }
        cout << ans << endl;
    }
    return 0;
}
/* Author: Hridoy Barua (CS Instructor Phitron) */
```

Double Discount :

- **Question Analysis:**

আমার কাছে **N food items** আছে।

i-th item এর **cost A_i rupees** (**A_i is an even integer**), এবং **tastiness B_i** .

আমাকে **exactly 2 items** কিনতে হবে।

এই **2 টা items** এর মধ্যে যে **item** টা **cost** বেশি সেটা তে আমি **discount** পাব।

- **Discount 50% or 100 rupees, whichever is smaller.**

অর্থাৎ - আমি **discount calculate** করব, এখন যেই ২ - টার মধ্যে যেটার **discount** কম সেটা আমি পাব।

এখন আমার কাছে **K rupees** আছে। আমাকে এমনভাবে ২টা **item choose** করতে হবে, যাতে আমার **2 টা item** এর **cost K** এর মধ্যেই হয়ে যায়, আর এদের **tastiness** এর **sum** টাও যেন **maximum** হয়।

- **Observation:**

- এখানে **Constraint** কম। **$1 \leq N \leq 100$**
- তারমানে **Brute Force Solution possible**
- আমি **nested loop** চালাতে পারব।
- যখন **$A_i \geq 100$** হবে তখনই কেবল আমি **100 discount** টা **use** করতে পারব।
- আর **50%** মানে **$A_i/2$**

- **Implementation:**

- **nested loop** চালাব।
- **Minimum discount** টা **calculate** করব।
- বড় **cost** টার উপর **discount apply** করব।
- একটা **ans variable maintain** করব।
- **Tastiness** এর **sum maximum** হলে, **ans variable update** করব।

- Code:

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long

int main(){
    int tc=1;
    cin >> tc;
    while(tc--){
        int n,k;
        cin >> n >> k;
        vector<int> a(n), b(n);
        for(int i=0; i<n; i++) cin >> a[i];
        for(int i=0; i<n; i++) cin >> b[i];

        int ans = 0;
        for(int i=0; i<n; i++){
            for(int j=i+1; j<n; j++){
                int small=a[i], boro=a[j];
                if(small>boro) swap(small, boro);

                int discount = boro/2;
                if(boro>=100){
                    discount = min(discount, 100);
                }

                boro-=discount;
                if(small+boro<=k) ans = max(ans, b[i]+b[j]);
            }
        }
        cout << ans << endl;
    }
    return 0;
}

/* Author: Hridoy Barua (CS Instructor Phitron) */
```

Target Temperature :

● Question Analysis:

আমার কাছে **N** চুলা আছে। শুরুতে - এদের **temperature 0**.

প্রতি **minute** - এ -

- আমি **at most** ১ টা চুলা **choose** করতে পারব, এবং এর **Temperature 0** দিয়ে **reset** করে দিতে পারি। **at most** ১ মানে - হয় যেকোনো ১টা **choose** নাহলে কোনটায় না।
- তারপর প্রতিটা চুলার **temperature 1** করে **increase** হবে।

এখন, **i-th** চুলা **ready** হবে যখন, এর **temperature exactly Bi** হবে।

আমাকে বলতে হবে। **operation** গুলো করে , আমি সব চুলাকে একসাথে **ready** করতে পারব কিনা। পারলে **Yes**, নাহলে - **No**.

● Observation:

- এখানে **maximum element** টা কিন্তু **reset** করার কোন দরকার নেই।
- বাকি এর থেকে ছোট সব **element** - গুলো **increase** হতে হতে **maximum** টা যখন সমান হবে, তখনি হবে।
- নিচের **testcase** গুলো দিয়ে আমরা একটু **dry run** করি।

Bi ->	1	2	3	4	4
Initial value ->	0	0	0	0	0
min-1	1	1	1	1	1
min-2	2	2	0+1	2	2
min-3	3	0+1	2	3	3
min-4	0+1	2	3	4	4

Finally:	1	2	3	4	4

- তাহলে আমরা যা দেখলাম, এখানে **maximum element** টা কে আমরা **touch** করি নাই একদম। আর এটা যতগুলোই থাক, ব্যাপার না।
- বাকি **value** গুলোর ক্ষেত্রে আমরা **descending order** এ **reset** করছি।

- তারমানে এটা **possible**. কিন্তু কখন **possible** না?
- এই **testcase** টা **same vabe** নিজে কর।

2 3 3 4

- দেখবে, 3 - দুইটার মধ্যে **clash** হচ্ছে। যাতে বুঝা যায়, **maximum element** ছাড়া অন্য কোন **element** ১ এর বেশিবার থাকতে পারবে না।

- **Implementation:**

- **Frequency calculate** করার জন্য **map use** করতে পারি।
- **Maximum element calculate** করার জন্য, **mx variable maintain** করতে পারি।
- যদি, আমার **frequency > 1 and element != mx** হয়, তাহলেই **possible** না।

- Code:

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long

int main(){
    int tc=1;
    cin >> tc;
    while(tc--){
        int n;
        cin >> n;
        int mx = 0;
        map<int,int> mp;
        for(int i=0; i<n; i++){
            int x;
            cin >> x;
            mx = max(mx, x);
            mp[x]++;
        }

        bool flag = true;
        for(auto [val,cnt]: mp){
            if(val!=mx && cnt>1){
                flag = false;
                break;
            }
        }

        if(flag) cout << "Yes\n";
        else cout << "No\n";

    }

    return 0;
}

/* Author: Hridoy Barua (CS Instructor Phitron) */
```